Dynamic Programming is mainly an optimization over plain recursion.

Wherever we see a recursive solution that has repeated calls for the same inputs, we can optimize it using Dynamic Programming.

The idea is to simply store the results of subproblems so that we do not have to re-compute them when needed later. This is known as **Overlapping Subproblems**.

Optimal Substructure

A problem exhibits optimal substructure if an optimal solution to the problem contains optimal solutions of the subproblems. That means we can solve subproblems and build up the solutions to solve larger problems.

This simple optimization reduces time complexities from exponential to polynomial.

Steps to solve a dynamic programming problem:

1. **Identifying if it is a Dynamic programming problem.**
   All dynamic programming problems satisfy the overlapping subproblems property and most of the problems also satisfy the optimal substructure property. Once we observe these properties in a given problem, one can apply DP.
2. **Deciding the state.**
   Problems with dynamic programming are mostly concerned with the state and its transition. This phase must be carried out with extreme care because the state transition depends on the state definition selected.
   State:
   A state is a collection of parameters that can be used to specifically describe a given position in the given problem. To minimise state space, this set of parameters has to be as compact as feasible.
3. **Formulating state and transition relationships.**
   It is the most difficult part of solving a DP problem as it requires observation, practice and intuition.
4. **Using Top-Down or Bottom-Up approach to solve the problem.**
   Simply storing the state solution will allow to access it from memory the next time that state is needed.

Ways to solve a DP problem:

There are 2 ways to solve a DP problem; Memoization and Tabulation.

Tabulation	Memoization
It is a Bottom-Up approach (iterative approach).	It is a Top-Down approach (recursive approach).
State transition relation is difficult to think.	State Transition relation is easy to think.
It is fast, as we directly access previous states from the table.	It is slow due to a lot of recursive calls and return statements.
Code gets complicated when a lot of conditions are required.	Code is easy and less complicated.
Starting from either the first entry or the last entry, all entries of the table are filled one by one.	Unlike the Tabulated version, all entries of the table are not necessarily filled in serial order. The table is filled on demand.